P0009r6 : `mdspan`: A Non-Owning Multidimensional Array Reference

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0009r6
Date: 2018-05-07
Reply-to: hedwards@nvidia.com
Author: H. Carter Edwards
Contact: hedwards@nvidia.com
Author: Bryce Adelstein Lelbach
Contact: blelbach@nvidia.com
Author: Daniel Sunderland
Contact: dsunder@sandia.gov
Author: David Hollman
Contact: dshollm@sandia.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Mauro Bianco
Contact: mbianco@cscs.ch
Author: Ben Sander
Contact: ben.sander@amd.com
Author: Athanasios Iliopoulos
Contact: athanasios.iliopoulos@nrl.navy.mil
Author: John Michopoulos
Contact: john.michopoulos@nrl.navy.mil
Author: Daniel Sunderland
Contact: dsunder@sandia.gov

Audience: Library Working Group (LWG)
URL: https://github.com/ORNL/cpp-proposals-pub/P0009/P0009.rst

1 Revision History

1.1 P0009r0 : Pre 2015-10-Kona Mailing

Original non-owning multidimensional array reference (`view`) paper with motivation, specification, and examples.

1.2 P0009r1 : Pre 2016-02-Jacksonville Mailing

LEWG review at 2015-10-Kona.

LEWG Poll: What should this feature be called?

Name	#
`view`	5
`span`	9
`array_ref`	6
`slice`	6
`array_view`	6
`ref`	0
`array_span`	7
`basic_span`	1
`object_span`	3

`field`	0

LEWG Poll: Do we want 0-length static extents?

SF	F	N	A	SA
3	4	2	3	0

LEWG POLL: Do we want the language to support syntaxes like X[3][][][5]?

Syntax	#
`view<int[3][0][][5], property1>`	12
`view<int, dimension<3, 0, dynamic_extent, 5>, property1>`	4
`view<int[3][0][dynamic_extent][5], property1>`	5
`view<int, 3, 0, dynamic_extent, 5, property1>`	4
`view<int, 3, 0, dynamic_extent, 5, properties<property1>>`	2
`view<arr<int, 3, 0, dynamic_extent, 5>, property1>`	4
`view<int[3][0][][5], properties<property1>>`	9

LEWG POLL: Do we want the variadic property list in template args (either raw or in `properties<>`)? Note there is no precedence for this in the library.

SF	F	N	A	SA
3	6	3	0	0

LEWG POLL: Do we want the per-view bounds-checking knob?

SF	F	N	A	SA
3	4	1	2	1

Changes from P0009r0:

- Renamed `view` to `array_ref`.
- How are users allowed to add properties? Needs elaboration in paper.
- `view<int[][][]>::layout` should be named.
- Rename `is_regular` (possibly to `is_affine`) to avoid overloading the term with the `Regular` concept.
- Make static span(), operator(), constructor, etc variadic.
- Demonstrate the need for improper access in the paper.
- In `operator()`, take integral types by value.

1.3 P0009r2 : Pre 2016-06-Oulu Mailing

LEWG review at 2016-02-Jacksonville.

Changes from P0009r1:

- Adding details for extensibility of layout mapping.
- Move motivation, examples, and relaxed incomplete array type proposal to separate papers. - P0331: Motivation and Examples for Polymorphic Multidimensional Array. - P0332: Relaxed Incomplete Multidimensional Array Type Declaration.

1.4 P0009r3 : Post 2016-06-Oulu Mailing

LEWG review at 2016-06-Oulu

LEWG did not like the name `array_ref`, and suggested the following alternatives: - `sci_span` - `numeric_span` - `multidimensional_span` - `multidim_span` - `mdspan` - `md_span` - `vla_span` - `multispan` - `multi_span`

LEWG Poll: Are member `begin()`/`end()` still good?

SF	F	N	A	SA
0	2	4	3	1

LEWG Poll: Want this proposal to provide range-producing functions outside array_ref?

SF	F	N	A	SA
0	1	3	2	3

LEWG Poll: Want a separate proposal to explore iteration design space?

SF	F	N	A	SA
9	1	0	0	0

Changes from P0009r2:

- Removed iterator support; a future paper will be written on the subject.
- Noted difference between multidimensional array versus language's array-of-array-of-array...
- Clearly describe requirements for the embedded type aliases (`element_type`, `reference`, etc).
- Expanded description of how the variadic properties list would work.
- Stopped allowing *array_ref<T[N]>* in addition to *array_ref<extents<N>>*.
- Clarified domain, codomain, and domain -> codomain mapping specifications.
- Consistently use *extent* and *extents* for the multidimensional index space.

1.5 P0009r4 : Pre 2017-11-Albuquerque Mailing

LEWG review at 2017-03-Kona meeting

LEWG review of P0546r1 at 2017-03-Kona meeting

LEWG Poll: Should we have a single template that covers both single and multi-dimensional spans?

SF	F	N	A	SA
1	6	2	6	3

Changes from P0009r3:

- Align with *P0122r5 ``span``* proposal <https://wg21.link/P0122r5>`_.
- Rename to `mdspan`, multidimensional span, to align with `span`.
- Move preferred array extents mechanism to appendix.
- Expose codomain as a `span`.
- Add layout mapping concept.

1.6 P0009r5 : Pre 2018-03-Jacksonville Mailing

LEWG review of P0009r4 at 2017-11-Albuquerque meeting

LEWG Poll: We should be able to index with span<int type[N]> (in addition to array).

SF	F	N	A	SA
2	11	1	1	0

Against comment - there is not a proven needs for this feature.

LEWG Poll: We should be able to index with 1d `mdspan`.

SF	F	N	A	SA
0	8	7	0	0

LEWG Poll: We should put the requirement on "rank() <= N" back to "rank() == N".

Unanimous consent

LEWG Poll: With the editorial changes from small group, plus the above polls, forward this to LWG for Fundamentals v3.

Unanimous consent

Changes from P0009r4:

- Removed *nullptr* constructor.
- Added constexpr to indexing operator.
- Indexing operator requires that `rank() == sizeof...(indices)`.
- Fixed typos in examples and moved them to appendix.
- Converted note on how extentions to access properties may cause reference to be a proxy type to an "see below" to make it normative.

1.6.1 Related Activity

Related LEWG review of P0546r1 at 2017-11-Albuquerque meeting

LEWG Poll: `span` should specify the dynamic extent as the element type of the first template parameter rather than the (current) second template parameter

SF	F	N	A	SA
5	3	2	2	0

LEWG Poll: `span` should support the addition of access properties variadic template parameters

SF	F	N	A	SA
0	10	1	5	0

Authors agreed to bring a separate paper (P0900) discussing how the variadic properties will work.

1.7 P0009r6 : Pre 2018-06-Rapperswil Mailing

P0009r5 was not taken up at 2018-03-Jacksonville meeting. Related LEWG review of P0900 at 2018-03-Jacksonville meeting

LEWG Poll: We want the ability to customize the access to elements of span (ability to restrict, etc):

```
span<T, N, Accessor=...>
```

SF	F	N	A	SA
1	1	1	2	8

LEWG Poll: We want the customization of basic_mdspan to be two concepts `Mapper` and `Accessor` (akin to `Allocator` design).

```
basic_mdspan<T, Extents, Mapper, Accessor>
mdspan<T, N...>
```

SF	F	N	A	SA
3	4	5	1	0

LEWG Poll: We want the customization of basic_mdspan to be an arbitrary (and potentially user-extensible) list of properties.

```
basic_mdspan<T, Extents, Properties...>
```

SF	F	N	A	SA
1	2	2	6	2

Changes from P0009r5 due to related LEWG reviews:

- Replaced variadic property list with *extents*, *layout mapping*, and *accessor* properties.
- Incorporated P0454r1. - Added accessor policy concept. - Renamed `mdspan` to `basic_mdspan`. - Added a `mdspan` alias to `basic_mdspan`.

2 Description

The proposed polymorphic multidimensional array reference (`mdspan`) defines types and functions for mapping indices from the **domain**, a **multidimensional index space**, to the **codomain**, elements of a contiguous span of objects. A multidimensional

index space is defined as the Cartesian product of integer extents, **[0..N0) X [0..N1) X [0..N2) ...**. An `mdspan` has two policies: the **layout mapping** and the **accessor**. The layout mapping specifies the formula, and properties of the formula, for mapping a multi-index from the domain to an element in the codomain. The accessor is an extension point that allows modification of how elements are accessed. For example, the Accessors paper (P0367) proposed a rich set of potential access properties.

A multidimensional array is not an array-of-array-of-array-of-array...

The multidimensional array abstraction has been fundamental to numerical computations for over five decades. However, the C/C++ language provides only a one dimensional array abstraction which can be composed into array-of-array-of-array... types. While such types have some similarity to multidimensional arrays they do not provide adequate multidimensional array functionality of this proposal. Two critical functionality differences are (1) multiple dynamic extents and (2) polymorphic mapping of multi-indices to element objects.

Optimized Implementation of Layout Mapping

The layout mapping of a multi-index intended to be an O(1) *constexpr* operation that is trivially inlined and optimized. Note that FORTRAN compilers' optimizations include loop invariant code motion, including partial evaluation of multi-index layout mappings when indices are loop-invariant.

3 Multidimensional Array and Subarray Proposal

3.1 Add to same section and header as span

```
namespace std {

  inline constexpr ptrdiff_t dynamic_extent = -1; // as per span

  // Multidimensional extents:
  template< ptrdiff_t ... StaticExtents >
  class extents ;

  template< ptrdiff_t ... LHS , ptrdiff_t ... RHS >
  constexpr bool operator == ( extents<LHS...> const & lhs , extents<RHS...> const & rhs ) ;

  template< ptrdiff_t ... LHS , ptrdiff_t ... RHS >
  constexpr bool operator != ( extents<LHS...> const & lhs , extents<RHS...> const & rhs ) ;

  // Layout mapping of multidimensional extents:
  class layout_right ;
  class layout_left ;
  class layout_stride ;

  // Multidimensional span:
  template <typename ElementType,
            typename Extents,
            typename LayoutPolicy = layout_right,
            typename AccessorPolicy = accessor_basic>
  class basic_mdspan;

  template <class T, ptrdiff_t... Extents>
  using mdspan = basic_mdspan<T, extents<Extents...>>;

  // return type of subspan free function is an mdspan
  template< typename ElementType , typename Extents, typename LayoutPolicy ,
            typename AccessorPolicy , typename ... SliceSpecifiers >
    // for exposition only:
    detail::subspan_deduction_t< basic_mdspan<ElementType, Properties...>,SliceSpecifiers...>
  subspan( basic_mdspan< ElementType, Extents , LayoutPolicy , AccessorPolicy > const & , SliceSpecifiers ... ) noexcept;

  // tag supporting subspan
  struct all_type {};
  inline constexpr all_type all = all_type{};
}
```

The `basic_mdspan` class maps a multi-index within a multi-index **domain** to a reference an element the **codomain** span.

The `subspan` free function generates an `basic_mdspan` with a domain contained within the input `basic_mdspan` domain and codomain contained within the input `basic_mdspan` codomain.

The *detail::subspan_deduction_t* template class is not proposed and appears for exposition only. An implementation metafunction of this form is necessary to deduce the specific `basic_mdspan` return type of the `subspan` function.

3.2 Class template `basic_mdspan`

```
namespace std {

template< typename ElementType , typename Extents , typename LayoutPolicy , typename AccessorPolicy >
class basic_mdspan {
public:
  // Domain and codomain types

  using layout   = LayoutPolicy;
  using mapping  = typename layout::mapping<Extents>;
  using accessor = typename AccessorPolicy::accessor<ElementType>;

  using element_type    = typename accessor::value_type;
  using value_type      = remove_cv_t<element_type>;
  using index_type      = ptrdiff_t ;
  using difference_type = ptrdiff_t ;
  using pointer         = typename accessor::pointer;
  using reference       = typename accessor::reference;

  // Standard constructors, assignments, and destructor

  ~basic_mdspan() noexcept ;

  constexpr basic_mdspan() noexcept ;
  constexpr basic_mdspan(basic_mdspan&&) noexcept = default ;
  constexpr basic_mdspan(basic_mdspan const&) noexcept = default ;
  basic_mdspan& operator=(basic_mdspan&&) noexcept = default ;
  basic_mdspan& operator=(basic_mdspan const&) noexcept = default ;

  // Constructor and assignment for assignable basic_mdspan

  template <typename UT, typename UExt, typename ULayPol, typename UAccPol>
  constexpr basic_mdspan(const basic_mdspan<UT, UExt, ULayPol, UAccPol>& other) noexcept;

  template <typename UType, typename ... UProp>
  constexpr basic_mdspan& operator=(const basic_mdspan<UT, UExt, ULayPol, UAccPol>& other) noexcept;

  // Wrapping constructors

  template< class ... IndexType >
  explicit constexpr basic_mdspan(pointer, IndexType ... DynamicExtents ) noexcept;

  template< class ... IndexType >
  explicit constexpr basic_mdspan(span<element_type>, IndexType ... DynamicExtents ) noexcept;

  template< class IndexType , size_t N >
  explicit constexpr basic_mdspan(pointer, array<IndexType,N> const & DynamicExtents ) noexcept ;

  template< class IndexType , size_t N >
  explicit constexpr basic_mdspan(span<element_type>, array<IndexType,N> const & DynamicExtents ) noexcept ;

  // Mapping domain multi-index to access codomain element

  constexpr reference operator[]( index_type ) const noexcept;

  template< class ... IndexType >
  constexpr reference operator()( IndexType ... indices ) const noexcept;

  template< class IndexType , size_t N >
  constexpr reference operator()( array<IndexType,N> const & indices ) const noexcept;

  // Observers of the domain multi-index space

  static constexpr int rank() noexecept ;
  static constexpr int rank_dynamic() noexecept ;

  static constexpr index_type static_extent(int) noexecept ;

  constexpr index_type extent(int) const noexecept ;

  constexpr index_type size() const noexecept ;

  // Observers of the codomain:

  constexpr span<element_type> span() const noexecept ;
```

```
  template< class ... IndexType >
  static constexpr index_type required_span_size( IndexType ... DynamicExtents );

  template< class ... IndexType , size_t N >
  static constexpr index_type required_span_size( array<IndexType,N> const & DynamicExtents );

  // Observers of the mapping : domain -> codomain

  static constexpr bool is_always_unique     = mapping::is_always_unique ;
  static constexpr bool is_always_contiguous = mapping::is_always_contiguous ;
  static constexpr bool is_always_strided    = mapping::is_always_strided ;

  constexpr bool is_unique() const noexcept ;
  constexpr bool is_contiguous() const noexcept ;
  constexpr bool is_strided() const noexcept ;

  constexpr index_type stride(int) const ;

private:
  // exposition only
  accesor      access;
  mapping      map;
  pointer_type ptr;
};

}
```

3.2.1 Template Arguments

`ElementType` is a (possibly cv-qualified) non-array object type denoting the element type of the array.

`Extents` is an instantiation of `extents`.

`LayoutPolicy` is a type that satisfies the `LayoutPolicy` requirements.

`AccessorPolicy` is a type that satisfies the `AccessorPolicy` requirements.

3.2.2 Domain Observers

The multi-index domain space is the Cartesian product of the extents: `[0..extent(0)) X [0..extent(1)) X ... X [0..extent(rank() - 1))`. Each extent may be statically (at compile time) or dynamically (at runtime) specified.

`static constexpr int rank();`

Returns: Rank of the multi-index domain.

`static constexpr int rank_dynamic();`

Returns: number of extents that are dynamic.

`static constexpr index_type static_extent(int r);`

Requires: `0 <= r`.

Returns: If `0 <= r < rank()` the statically specified extent. A statically declared extent of `dynamic_extent` denotes that the extent is dynamic. If `rank() <= r` then `static_extent(r) == 1`.

`constexpr index_type extent(int r) const ;`

Requires: `0 <= r`.

Returns: If `0 <= r < rank()` the extent of coordinate `r`. If `rank() <= r` then `extent(r) == 1`.

`constexpr index_type size() const ;`

Returns: Product of `extent(r)` where `0 <= r < rank()`.

3.2.3 Codomain Observers

Not all elements of the codomain may be accessible through the layout mapping; i.e., the range of the mapping is contained within the codomain but may not be equal to the codomain.

`constexpr span<element_type> span() const ;`

Returns: An `span` for the codomain.

```
template< class ... IndexType >
static constexpr index_type required_span_size( IndexType ... DynamicExtents );
```

Requires:

- `rank_dynamic() == sizeof...(DynamicExtents)`.
- `is_integral_type_v<IndexType>...`.
- Let `DynamicExtents[ith]` denote the ith coordinate of `DynamicExtents...` `0 <= DynamicExtents[ith]` for `0 <= ith < rank_dynanic()`.

Returns: The minimum size of the codomain to support the multi-index domain defined by the merging of `DynamicExents` with `StaticExtents`.

```
template< class ... IndexType , size_t N >
static constexpr index_type required_span_size( array<IndexType,N> const & DynamicExtents );
```

Requires:

- `rank_dynamic() == N`
- `is_integral_type_v<IndexType>...`
- `0 <= DynamicExtents[ith]` for `0 <= ith < rank_dynanic()`

Returns: The minimum size of the codomain to support the multi-index domain defined by the merging of `DynamicExents` with `StaticExtents`.

3.2.4 Constructors, assignments, destructor

```
constexpr basic_mdspan();
```

Effect: Construct a *null* `mdspan` with codomain `span() == span<element_type>()` and `extent(r) == 0` for all dynamic extents.

```
template <typename UT, typename UExt, typename ULayPol, typename UAccPol>
`` constexpr basic_mdspan(const basic_mdspan<UT, UExt, ULayPol, UAccPol>& other) noexcept;``
template <typename UType, typename ... UProp>
`` constexpr basic_mdspan& operator=(const basic_mdspan<UT, UExt, ULayPol, UAccPol>& other) noexcept;``
```

Requires: Given `using V = basic_mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>` and `using U = basic_mdspan<UT, UExt, ULayPol, UAccPol>` then

`is_assignable_v<typename V::pointer, typename U::pointer> == true`,
`V::rank() == U::rank()`,
`V::static_extent(r) == U::static_extent(r)` or `V::static_extent(r) == dynamic_extent` for `0 <= r < V::rank()`,
compatibility of layout mapping

Effect: `* this` has equal domain, equal codomain, and equivalent mapping.

```
template< class ... IndexType >
constexpr basic_mdspan( pointer ptr , IndexType ... DynamicExtents) noexcept
```

Requires:

- `sizeof...(DynamicExtents) == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- The ith coordinate of `DynamicExtents...`, denoted as `DynamicExtents[ith]`, is `0 <= DynamicExtents[ith]`.
- `[ ptr , ptr + required_span_size(DynamicExtents...) )`, shall be a valid contiguous span of elements.

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents...` and codomain equal to `span<element_type>( ptr , required_span_size(DynamicExtents...) )`

```
template< class IndexType , size_t N >
constexpr basic_mdspan( pointer ptr , array<IndexType,N> const & DynamicExtents) noexcept
```

Requires:

- `N == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- `0 <= DynamicExtents[ith]`

- The span of elements denoted by `[ ptr , ptr + required_span_size(DynamicExtents) )`, shall be a valid contiguous span of elements.

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents[ith].` and codomain equal to `span<element_type>( ptr , required_span_size(DynamicExtents) )`

```
template< class ... IndexType >
constexpr basic_mdspan( span<element_type> s , IndexType ... DynamicExtents) noexcept
```

Requires:

- `sizeof...(DynamicExtents) == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- The ith coordinate of `DynamicExtents...`, denoted as `DynamicExtents[ith]`, is `0 <= DynamicExtents[ith]`
- `required_span_size(DynamicExtents...) <= s.size()`

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents...` and codomain equal to `span<element_type>( ptr , required_span_size(DynamicExtents...) )`

```
template< class IndexType , size_t N >
constexpr basic_mdspan( span<element_type> s , array<IndexType,N> const & DynamicExtents) noexcept
```

Requires:

- `N == rank_dynamic()`
- `is_integral_type_v<IndexType>...`
- `0 <= DynamicExtents[ith]`
- `required_span_size(DynamicExtents) <= s.size()`

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to `DynamicExtents[ith]` and codomain equal to `span<element_type>( ptr , required_span_size(DynamicExtents[ith]) )`

3.2.5 Mapping domain multi-index to access elements in the codomain

```
template< class ... IndexType >
reference operator()( IndexType ... indices ) const noexcept
```

Requires: `indices` is a multi-index in the domain:

- `rank() == sizeof...(IndexType)`
- The ith coordinate of `indices...`, denoted as `indices[ith]`, is in the domain: `0 <= indices[ith] < extent(ith)`.

Returns: A `reference` to the element mapped to by `indices...`.

```
reference operator[]( index_type index ) const noexcept
```

Effects: Equivalent to `return (*this)(index);`

```
template< class IndexType , size_t N >
reference operator()( array<IndexType,N> const & indices ) const noexcept
```

Effects: Equivalent to `(*this)(indices[0], ... indices[N-1]);`.

3.2.6 Layout Mapping Observers

```
static constexpr bool is_always_unique =
constexpr bool is_unique() const noexcept ;
```

A layout mapping is *unique* if each multi-index in the domain is mapped to a unique element in the codomain.

```
static constexpr bool is_always_contiguous =
constexpr bool is_contiguous() const noexcept ;
```

A layout mapping is *contiguous* if the codomain elements accessed through the layout mapping form a contiguous span.

A layout mapping that is both unique and contiguous is *bijective*, and as a consequence `size() == span().size()`.

```
static constexpr bool is_always_strided =
constexpr bool is_strided() const noexcept ;
```

A *strided* layout has constant striding between multi-index coordinates. Let `A` be an `mdspan` and `indices_V...` and

`indices_U...` be multi-indices in the domain space such that all coordinates are equal except for the *ith* coordinate where `indices_V[ith] = indices_U[ith] + 1`. Then `stride(ith) = distance(& A(indices_V...) - & A( indices_U... )` is constant for all valid coordinates.

```
template< typename IntegralType >
constexpr index_type stride( IntegralType index ) const noexcept
```

Requires: `is_strided()`.

Returns: When `r < rank()` the distance between elements when the index of coordinate `r` is incremented by one, otherwise 0.

3.3 Class template `extents`

An extents objects defines the lengths of the dimensions of an `mdspan`.

```
namespace std {

template< ptrdiff_t ... StaticExtents >
class extents {
public:

  using index_type = ptrdiff_t ;

  // Constructors/assignment/destructor

  ~extents() = default ;
  constexpr extents();
  constexpr extents(extents const &) = default ;
  constexpr extents(extents &&) = default ;
  extents & operator = (extents const &) noexcept = default ;
  extents & operator = (extents &&) noexcept = default ;

  template< class ... IndexType >
  constexpr extents( IndexType ... DynamicExtents ) noexcept ;

  // Observers of the index space domain:

  static constexpr int rank() noexcept ;
  static constexpr int rank_dynamic() noexcept ;

  static constexpr index_type static_extent(int) noexcept ;

  constexpr index_type extent(int) const noexcept ;
};

}
```

The multi-index domain space is the Cartesian product of the extents: `[0..extent(0)) X [0..extent(1)) X ... X [0..extent(rank() - 1))`. Each extent may be statically (at compile time) or dynamically (at runtime) specified.

```
template< ptrdiff_t StaticExtents... >
```

Requires: `0 <= StaticExtent` or `dynamic_extent == StaticExtent`

```
static constexpr int rank();
```

Returns: Rank of the multi-index domain, `sizeof...(StaticExtents)`.

```
static constexpr int rank_dynamic();
```

Returns: number of `StaticExtents...` that are `dynamic_extent`.

```
static constexpr index_type static_extent(int r);
```

Requires: `0 <= r`.

Returns: If `0 <= r < rank()` the statically specified extent. A statically declared extent of `dynamic_extent` denotes that the extent is dynamic. If `rank() <= r` then `static_extent(r) == 1`.

```
constexpr index_type extent(int r) const ;
```

Requires: `0 <= r`.

Returns: If `0 <= r < rank()` the extent of coordinate `r`. If `rank() <= r` then `extent(r) == 1`.

```
template< ptrdiff_t ... LHS , ptrdiff_t ... RHS >
constexpr bool operator == ( extents<LHS...> const & lhs , extents<RHS...> const & rhs ) ;
```

Returns: `lhs.rank() == rhs.rank()` and `lhs.extent(r) == rhs.extent(r)` for `0 <= r < lhs.rank()`.

```
template< ptrdiff_t ... LHS , ptrdiff_t ... RHS >
constexpr bool operator != ( extents<LHS...> const & lhs , extents<RHS...> const & rhs ) ;
```

Returns: `! ( lhs == rhs )`

3.4 subspan

```
template <typename ElementType, typename Extents,
          typename LayoutPolicy, typename AccessorPolicy>
// for exposition only:
detail::subspan_deduction_t<basic_mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>, SliceSpecifiers...>
subspan(const basic_mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>& U , SliceSpecifiers... slices) noexcept;
```

The `detail::subspan_deduction_t` is for exposition only to indicate that the implementation is required to deduce the resulting `basic_mdspan` type from `U` and `slices...`.

Let the *ith* element of `slices...` be denoted by `slices[ith]`.

Let an *integral range* be denoted by any of the following.

- an `initializer_list<T>` of integral type `T` and size 2
- a `pair<T,T>` of integral type `T`
- a `tuple<T,T>` of integral type `T`
- an `array<T,2>` of integral type `T`
- `all` to denote the range `[0 .. U.extent(ith))`

If `slices[ith]` is an integral range then let `begin(slices[ith])` be the beginning of the integral range `end(slices[ith])` be the end of the integral range. If `slices[ith]` is an integral value then let `begin(slices[ith]) == slices[ith]` and `end(slices[ith]) == slices[ith]+1`.

Requires:

- `U.rank() == sizeof...(slices)`.
- Each element of the `slices...` pack is either an *integral range* or an *integral value*.
- `0 <= begin(slices[ith]) <= end(slices[ith]) <= U.extent(ith)`.

Returns: A `basic_mdspan V` with a domain contained within the domain of `U`, codomain contained within the codomain of `U`, `V.rank()` is the number of integral ranges in `slices...`, `U( begin(slices)... )` refers to the same codomain element refered to by the mapping the zero-index of `V`, each integral value in `slices...` contracts the corresponding extent of `U`.

[Note: An illustrative example

```
// given U.rank() == 4
void foo( basic_mdspan< ElementType , Extents , LayoutPolicy > const & U )
{
  auto V = subspan( U , make_pair(1,U.extent(0)-1) , 1 , make_pair(2,U.extent(2)-2 );
  assert( V.extent(0) == U.extent(0) - 2 );
  assert( V.extent(1) == U.extent(2) - 2 );
  assert( & V(0,0) == U(1,1,2,2) );
  assert( & V(1,0) == U(2,1,2,2) );
  assert( & V(0,1) == U(1,1,3,2) );
}
```

--end note]

3.4.1 Slice Specifier with Static Extent

The proposed `initializer_list`, `pair`, `tuple`, and `array` slice specifier types define dynamic extents. When the `all` slice specifier references a static extent then the subspan's corresponding extent should be static as well. When the extent of a slice specifier is statically known there should be a slice specifier type to explicitly express this knowledge. Such a static extent slice specifier type is to-be-done.

3.5 Layout Policy Requirements

A `basic_mdspan` maps multi-indices from the domain to reference elements in the codomain by composing a *layout mapping* with a span of elements. The layout mapping is an extension point such that an `basic_mdspan` may be instantiated with non-standard layout mappings.

3.5.1 Predefined, Standard Layout Policies

The `layout_right` property denotes the C/C++ standard multidimensional array index mapping where the right-most extent is stride one and strides increase right-to-left as the product of extents.

The `layout_left` property denotes the FORTRAN standard multidimensional array index mapping where the left-most extent is stride one and strides increase left-to-right as the product of extents.

The `layout_stride` property denotes a multidimensional array index mapping with arbitrary strides for each extent. This is the layout for subarrays that are not contiguous.

The three standard layouts have the following layout mapping traits.

`layout_right` ; i.e., the C/C++ standard layout

> `is_always_unique == true`
> `is_always_contiguous == true`
> `is_always_strided == true`
> When `0 < rank()` then `stride(rank()-1) == 1` .
> When `1 < rank()` then `stride(r-1) = stride(r) * extent(r)` for `0 < r < rank()` ..
>
> For rank-two arrays (a.k.a., matrices) this is also known as *row major* layout.

`layout_left` ; i.e., the FORTRAN standard layout

> `is_always_unique == true`
> `is_always_contiguous == true`
> `is_always_strided == true`
> When `0 < rank()` then `stride(0) == 1` .
> When `1 < rank()` then `stride(r) = stride(r-1) * extent(r-1)` for `0 < r < rank()` ..
>
> For rank-two arrays (a.k.a., matrices) this is also known as *column major* layout.

`layout_stride` ; i.e., an arbitrary **strided** layout

> `is_always_unique == false`
> `is_always_contiguous == false`
> `is_always_strided == true`

3.5.2 Concept for Extensible Layout Mapping

A *layout* class conforms to the following interface such that an `mdspan` can compose the layout mapping with its `mdspan` codomain element reference generation.

```
class layout_property /* exposition only */ {
public:

  template< ptrdiff_t ... StaticExtents >
  class mapping {
  public:

    // Domain types

    using index_type = ptrdiff_t ;

    // Constructors, copy, assignment, and destructor

    ~mapping() noexcept = default ;
    constexpr mapping() noexcept = default ;
    constexpr mapping(mapping const&) noexcept = default ;
    mapping& operator=(mapping const&) noexcept = default ;

    // Observers of domain

    static constexpr int rank() noexcept;
    static constexpr int rank_dynamic() noexcept;

    static constexpr index_type static_extent(int) noexcept;
```

```
    constexpr index_type extent(int) const noexcept;

    // Observers of the codomain: [0..required_span_size())

    constexpr index_type required_span_size() const noexcept;

    // Observers of the mapping from domain to codomain

    static constexpr bool is_always_unique     = /* unspecified */ ;
    static constexpr bool is_always_contiguous = /* unspecified */ ;
    static constexpr bool is_always_strided    = /* unspecified */ ;

    constexpr bool is_unique() const noexcept;
    constexpr bool is_contiguous() const noexcept;
    constexpr bool is_strided() noexcept;

    constexpr index_type stride(int) const noexcept;

    // Mapping domain index to access codomain element

    template< class ... IndexType >
    constexpr index_type operator()( IndexType ... indices ) const noexcept;
  };
};
```

`template< ptrdiff_t ... StaticExtents > class mapping`

Requires: Let `StaticExtents[ith]` be the ith member of the pack. `StaticExtents[ith] == dynamic_extent` or `0 <= StaticExtents[ith]`.

Effects: Defines the domain index space where `rank() == sizeof...(StaticExtents)` and each `StaticExtents[ith] == dynamic_extent` denotes that `ith` extent coordinate is a dynamic extent.

`constexpr mapping();`

Effects: If `static_extent(i) != dynamic_extent` then `extent(i) == static_extent(i)` otherwise `extent(i) == 0`.

`explicit constexpr mapping( index_type... ) noexcept;`
`explicit constexpr mapping(` *layout_property* `const&) noexcept;`

Constructors, assignment operators, and destructor requires and effects correspond to the corresponding members of `basic_mdspan`.

```
static constexpr int rank() noexcept;
static constexpr int rank_dynamic() noexcept;
constexpr index_type extent(int) const noexcept;
constexpr index_type static_extent(int) noexcept;

template < class ... IndexType >
static constexpr index_type required_span_size( IndexType ... DynamicExtents ) noexcept;
static constexpr index_type required_span_size( *layout_property* const & ) noexcept;

static constexpr bool is_always_unique     = /* unspecified */ ;
static constexpr bool is_always_contiguous = /* unspecified */ ;
static constexpr bool is_always_strided    = /* unspecified */ ;

constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() noexcept;

constexpr index_type stride(int) const noexcept;
```

Domain, codomain, and mapping observers requires and effects correspond to the corresponding members of `basic_mdspan`.

`constexpr index_type required_span_size() const noexcept;`

Returns: The upper bound of the codomain one dimensional index space `[0..required_span_size())`.

```
template< class ... IndexType >
constexpr index_type operator()(IndexType ... indices) const noexcept;
```

Requires: `rank() == sizeof...(indices)` and `0 <= indices[ith] < extent(ith)`.

Returns: Result of mapping `indices...` from the domain multidimensional index space to the codomain one

dimensional index space `[0..required_span_size())`.

3.6 Accessor Policy Requirements

An *accessor policy* is a class that contains an *accessor*, a nested template class.

An accessor takes a pointer to an array and an index and returns a reference to the element of the array at the given index. `basic_mdspan` is parameterized in terms of accessor.

An accessor fulfills the `DefaultConstructible` and `CopyAssignable` requirements.

In Table ❖:

- `AP` denotes an accessor policy.
- `A` denotes an accessor.
- `a` denotes a value of type `A`.
- `p` denotes a pointer of type `T`.
- `i` denotes an integer.

Expression	Return Type	Operational Semantics	Requirements/Note
`AP::accessor<T>`	`A`		
`A::pointer`	A `RandomAccessIterator`		*Requires:* This iterator type's value type shall be convertible to `T`.
`A::reference`			*Requires:* This type shall be convertible to `T`.
`a(p, i)`	`reference`	*Effects:* Equivalent to `return p[i];`	

3.7 Accessor Policies

```
struct accessor_basic
{
  template <typename T>
  struct accessor
  {
    using pointer      = T*;
    using reference    = T&;

    constexpr reference operator()(pointer p, ptrdiff_t i) const noexcept;
  };
};
```

```
template <typename Index>
constexpr reference operator()(pointer p, Index i) const noexcept;
```

Effects: Equivalent to `return p[i];`

4 Next Steps

- Wording editing as per guidance from LWG.
- Address verbose syntax when using `std::dynamic_extent` to denote dynamic extents; *e.g.*, perhaps alias to `std::dyn`.

5 Related Work

LEWG issue

Previous paper:

- N4355

P0860 : Access Policy Generating Proxy Reference

The `reference` type may be a proxy for accessing an `element_type` object. For example, the *atomic* `AccessorPolicy` in **P0860** defines `AccessorPolicy::accessor<T>::reference` to be `atomic_ref<T>` from **P0019**.

Related papers:

- **P0122** : span: bounds-safe views for sequences of objects The `mdspan` codomain concept of *span* is well-aligned with this paper.
- **P0367** : Accessors The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The `AccessPolicy` extension point in this proposal is intended to include such memory access properties.
- **P0331** : Motivation and Examples for Multidimensional Array
- **P0332** : Relaxed Incomplete Multidimensional Array Type Declaration
- **P0454** : Wording for a Minimal `mdspan` Included proposed modification of `span` to better align `span` with `mdspan`.
- **P0546** : Preparing `span` for the future Proposed modification of `span`
- **P0856** : Restric access property for `mdspan` and `span`
- **P0860** : atomic access policy for `mdspan`
- **P0900** : An Ontology of Properties for `mdspan`